

Tugas 11: Praktikum Mandiri 11 Machine Learning

Hisyam Wildan Alfath - 0110222206

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum ini bertujuan untuk menerapkan algoritma DBSCAN dalam proses clustering data sekolah di Indonesia dengan jumlah sampel besar. Tahapan analisis meliputi pembersihan data, konversi tipe numerik, normalisasi menggunakan RobustScaler, serta evaluasi korelasi dan visualisasi distribusi data. Parameter eps dan min_samples ditentukan melalui K-Distance Graph, kemudian proses clustering dilakukan pada sampel data yang telah dinormalisasi. Hasil menunjukkan bahwa DBSCAN berhasil membentuk beberapa cluster utama tanpa memerlukan jumlah cluster yang ditentukan sebelumnya. Nilai Silhouette Score sebesar ± 0.36 mengindikasikan kualitas cluster berada pada kategori cukup baik. Visualisasi PCA 2D serta plot interaktif memperlihatkan pemisahan pola yang jelas antara kelompok data dengan kepadatan tinggi dan rendah. Secara keseluruhan, metode DBSCAN efektif digunakan untuk mengidentifikasi pola persebaran sekolah pada data berukuran besar dan tidak berdistribusi normal.

1. Praktikum Mandiri 11

- Instalasi Library Folium

```
[1] ✓ 11s !pip install folium
```

Langkah ini dilakukan untuk menginstal library Folium, yaitu library Python yang digunakan untuk membuat visualisasi peta berbasis leaflet. Folium akan digunakan pada tahap awal untuk menampilkan sebaran titik lokasi sekolah di Indonesia. Dengan menjalankan perintah `!pip install folium`, environment Colab memastikan bahwa semua fungsi pemetaan dapat berjalan tanpa error sebelum proses analisis dilanjutkan.

- Instalasi Library Contextily

```
[2] ✓ 14s !pip install contextily
```

Perintah `!pip install contextily` digunakan untuk menambahkan library Contextily ke dalam environment, sehingga kamu dapat menampilkan basemap ketika

membuat visualisasi peta menggunakan GeoPandas. Instalasi ini penting karena Contextily menyediakan layer peta seperti tiles dari OpenStreetMap yang membuat tampilan persebaran titik pada peta menjadi lebih jelas dan informatif.

- Import Library untuk Analisis dan Clustering

```
[3]
✓ 5s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import RobustScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

Pada tahap ini dilakukan import berbagai library yang dibutuhkan untuk keseluruhan proses analisis, mulai dari manipulasi data menggunakan Pandas dan NumPy, visualisasi dengan Matplotlib dan Seaborn, hingga fungsi-fungsi penting dari Scikit-learn seperti RobustScaler untuk normalisasi, NearestNeighbors untuk menentukan eps, DBSCAN sebagai algoritma clustering, Silhouette Score untuk evaluasi kualitas cluster, dan PCA untuk reduksi dimensi. Semua library ini menjadi fondasi utama agar analisis clustering dapat berjalan dengan lengkap dan terstruktur.

- Menghubungkan Google Drive ke Google Colab

```
[4]
✓ 24s
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Langkah ini digunakan untuk melakukan mounting Google Drive ke dalam environment Colab sehingga file yang tersimpan di Drive, seperti dataset data.csv, dapat diakses dan digunakan langsung dalam proses analisis. Perintah `drive.mount('/content/drive')` membuat folder Drive tampil sebagai direktori lokal sehingga memudahkan pemanggilan dan penyimpanan data selama pengerjaan notebook.

- Membaca Dataset dari Google Drive

```
[5] ✓ 3s
df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 11/Praktikum dan Tugas Mandiri 11/data/data.csv")
df.head()
```

/tmp/ipython-input-766724893.py:1: DtypeWarning: Columns (6) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 11/Praktikum dan Tugas Mandiri 11/data/data.csv")
```

	province_id	province_name	city_id	city_name	district_id	district	npsn	school_name	stage	status	street_name	lat	long
0	250000	Papua	251800	Kab. Tolikara	251831	Kec. Gilubandu	60303704	SMP SATAP KAGI	SMP	N	Gilubandu	-3.415995	138.339704
1	250000	Papua	251800	Kab. Tolikara	251822	Kec. Wunin	60302267	SD YPPGI WURINERI	SD	S	Wunin	-3.479000	138.680000
2	250000	Papua	251800	Kab. Tolikara	251803	Kec. Bokondini	60302279	SMAN BOKONDINI	SMA	N	MAIRINI	-3.531731	138.658234
3	250000	Papua	251800	Kab. Tolikara	251814	Kec. Kuari	60302215	SD INPRES KUARI	SD	N	Kuari	-3.518100	138.529500
4	250000	Papua	251800	Kab. Tolikara	251802	Kec. Karubaga	60302283	SMAS YPPGI KARUBAGA	SMA	S	AMPERA	-3.440100	138.598800

Pada tahap ini dataset data.csv dibaca dari direktori Google Drive menggunakan `pd.read_csv()`. Setelah file berhasil dimuat ke dalam variabel `df`, perintah `df.head()` digunakan untuk menampilkan beberapa baris pertama sebagai verifikasi bahwa struktur data sudah terbaca dengan benar. Tampilan tersebut menunjukkan kolom-kolom penting seperti kode provinsi, nama kota, NPSN, jenjang sekolah, dan koordinat, yang nantinya menjadi dasar dalam proses analisis dan clustering menggunakan DBSCAN.

- Menampilkan Struktur Dataset dengan `df.info()`

```
[6] ✓ 0s
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 215372 entries, 0 to 215371
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   province_id      215372 non-null   int64
1   province_name    215372 non-null   object
2   city_id          215372 non-null   int64
3   city_name        215372 non-null   object
4   district_id      215372 non-null   int64
5   district         215372 non-null   object
6   npsn             215372 non-null   object
7   school_name      215372 non-null   object
8   stage            215372 non-null   object
9   status           215372 non-null   object
10  street_name      215370 non-null   object
11  lat              214222 non-null   float64
12  long             214222 non-null   float64
dtypes: float64(2), int64(3), object(8)
memory usage: 21.4+ MB
```

Pemanggilan `df.info()` digunakan untuk melihat struktur lengkap dataset, termasuk jumlah baris, jumlah kolom, tipe data setiap kolom, serta apakah terdapat nilai yang kosong. Dari hasil tersebut terlihat bahwa dataset berisi 215.372 baris dengan 13 kolom, terdiri dari campuran tipe data numerik dan objek. Informasi ini penting untuk menentukan kolom mana yang dapat digunakan dalam proses normalisasi dan clustering, sekaligus mengidentifikasi adanya nilai kosong pada kolom koordinat (`lat` dan `long`) yang perlu ditangani pada tahap selanjutnya.

- Menampilkan Daftar Nama Kolom Dataset

```
[7] df.columns
✓ 0s
Index(['province_id', 'province_name', 'city_id', 'city_name', 'district_id',
      'district', 'npsn', 'school_name', 'stage', 'status', 'street_name',
      'lat', 'long'],
      dtype='object')
```

Perintah `df.columns` digunakan untuk menampilkan seluruh nama kolom yang terdapat dalam dataset. Daftar ini membantu mengidentifikasi variabel mana yang bersifat numerik maupun kategorikal, sehingga memudahkan proses pemilihan fitur pada tahap normalisasi dan clustering. Dari hasil tampilan terlihat bahwa dataset memiliki kombinasi kolom kode wilayah, informasi sekolah, serta koordinat, yang nantinya hanya sebagian diambil untuk analisis DBSCAN sesuai kebutuhan.

- Pemeriksaan Missing Value pada Dataset

```
[8] df.isnull().sum()
✓ 0s
0
province_id    0
province_name  0
city_id        0
city_name      0
district_id    0
district       0
npsn           0
school_name    0
stage          0
status         0
street_name    2
lat           1150
long           1150
dtype: int64
```

Melalui perintah `df.isnull().sum()`, dilakukan pengecekan jumlah nilai kosong di setiap kolom. Hasilnya menunjukkan bahwa sebagian besar kolom memiliki data lengkap, kecuali `street_name` yang memiliki 2 nilai kosong serta `lat` dan `long` yang masing-masing memiliki 1.150 nilai kosong. Informasi ini penting untuk menentukan langkah penanganan data selanjutnya, terutama karena koordinat akan dibutuhkan pada visualisasi peta sehingga baris tanpa nilai latitude dan longitude perlu dipertimbangkan untuk dibersihkan atau dihapus sebelum analisis berbasis lokasi dilakukan.

- Menghapus Baris dengan Missing Value pada Kolom Penting

```
[9] df = df.dropna(subset=['street_name', 'lat', 'long']).copy()
[10] df.isnull().sum()
0
province_id    0
province_name  0
city_id        0
city_name      0
district_id    0
district       0
npsn           0
school_name    0
stage          0
status         0
street_name    0
lat            0
long           0
dtype: int64
```

Pada langkah ini dilakukan pembersihan data dengan menghapus baris yang memiliki nilai kosong pada kolom `street_name`, `lat`, dan `long`. Ketiga kolom tersebut dianggap penting terutama untuk kebutuhan analisis spasial dan identifikasi lokasi sekolah. Setelah proses `dropna`, pengecekan ulang menggunakan `df.isnull().sum()` menunjukkan bahwa seluruh kolom kini tidak memiliki missing value, sehingga dataset menjadi lebih bersih dan siap digunakan untuk analisis lanjutan tanpa risiko error akibat nilai kosong.

- Mengubah Variabel Kategori `stage` Menjadi Nilai Numerik

```
[11] df['stage'].unique()
array(['SMP', 'SD', 'SMA', 'SMK', 'SLB', 'SDLB', 'SMLB', 'SMPLB'],
      dtype=object)

[12] mapping_stage = {
    'SD': 1,
    'SDLB': 1,
    'SMP': 2,
    'SMPLB': 2,
    'SMA': 3,
    'SMLB': 3,
    'SMK': 4,
    'SLB': 5
}

df['stage'] = df['stage'].replace(mapping_stage)
/tmp/ipython-input-3675227720.py:12: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version.
df['stage'] = df['stage'].replace(mapping_stage)

[13] df['stage'].unique()
array([2, 1, 3, 4, 5])
```

Tahap ini dilakukan untuk mengonversi kolom `stage`, yang awalnya berisi kategori teks seperti `SD`, `SMP`, `SMA`, dan sebagainya, menjadi representasi numerik

agar dapat digunakan dalam analisis lebih lanjut. Proses dilakukan dengan membuat dictionary mapping_stage yang memetakan setiap kategori ke angka tertentu, kemudian menerapkannya menggunakan .replace(). Hasil pengecekan ulang melalui df['stage'].unique() menunjukkan bahwa seluruh nilai stage telah berhasil berubah menjadi angka, sehingga kolom ini menjadi lebih mudah diolah pada tahap pemodelan atau visualisasi berikutnya.

- Menampilkan Data Setelah Konversi Kolom stage

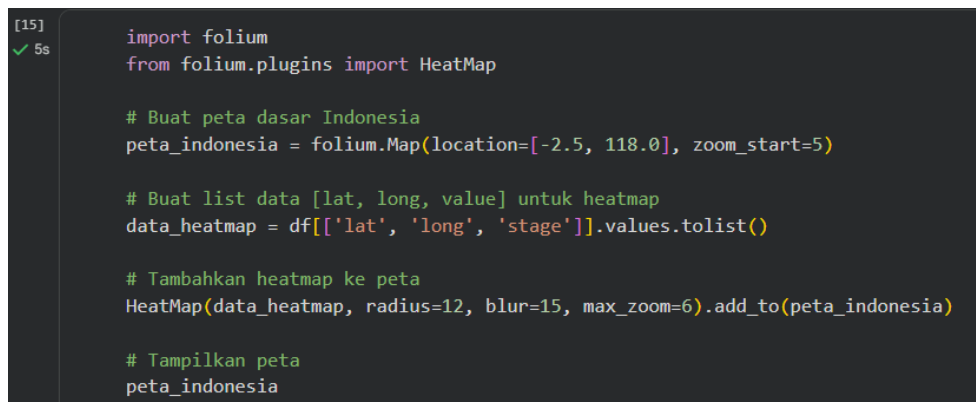


```
df.head()
```

	province_id	province_name	city_id	city_name	district_id	district	npsn	school_name	stage	status	street_name	lat	long
0	250000	Papua	251800	Kab. Tolikara	251831	Kec. Gilubandu	60303704	SMP SATAP KAGI	2	N	Gilubandu	-3.415995	138.339704
1	250000	Papua	251800	Kab. Tolikara	251822	Kec. Wunin	60302267	SD YPPGI WURINERI	1	S	Wunin	-3.479000	138.680000
2	250000	Papua	251800	Kab. Tolikara	251803	Kec. Bokondini	60302279	SMAN BOKONDINI	3	N	MAIRINI	-3.531731	138.658234
3	250000	Papua	251800	Kab. Tolikara	251814	Kec. Kuari	60302215	SD INPRES KUARI	1	N	Kuari	-3.518100	138.529500
4	250000	Papua	251800	Kab. Tolikara	251802	Kec. Karubaga	60302283	SMAS YPPGI KARUBAGA	3	S	AMPERA	-3.449100	138.598800

Setelah proses pengubahan nilai kategori pada kolom stage menjadi angka, langkah df.head() digunakan kembali untuk memastikan bahwa perubahan tersebut telah diterapkan dengan benar pada dataset. Dari tampilan awal baris data terlihat bahwa kolom stage kini berisi nilai numerik seperti 1, 2, dan 3 yang mewakili jenjang pendidikan tertentu. Dengan format numerik seperti ini, kolom tersebut menjadi lebih siap digunakan untuk analisis lanjutan, baik dalam pemodelan maupun visualisasi, tanpa risiko error akibat tipe data kategorikal.

- Visualisasi Heatmap Persebaran Sekolah di Indonesia



```
[15]
✓ 5s
import folium
from folium.plugins import HeatMap

# Buat peta dasar Indonesia
peta_indonesia = folium.Map(location=[-2.5, 118.0], zoom_start=5)

# Buat list data [lat, long, value] untuk heatmap
data_heatmap = df[['lat', 'long', 'stage']].values.tolist()

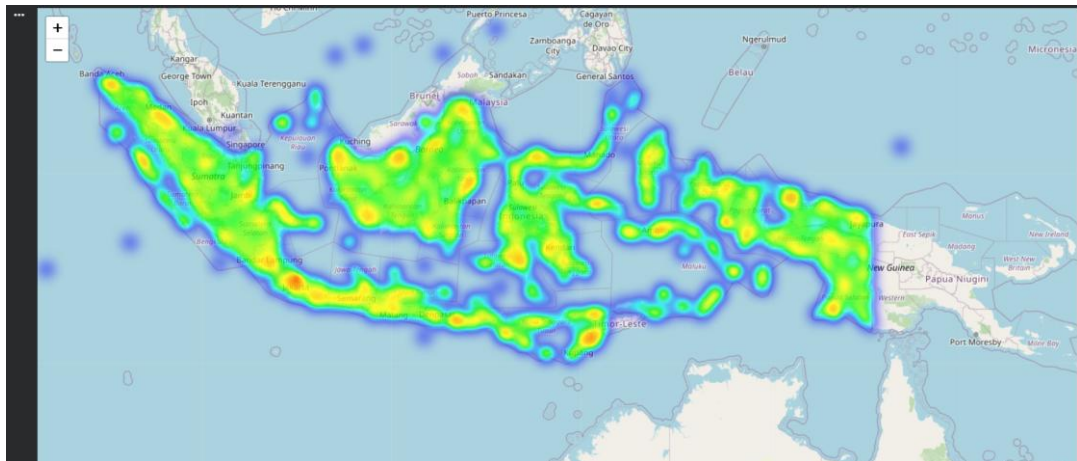
# Tambahkan heatmap ke peta
HeatMap(data_heatmap, radius=12, blur=15, max_zoom=6).add_to(peta_indonesia)

# Tampilkan peta
peta_indonesia
```

Kode ini digunakan untuk membuat visualisasi heatmap yang menunjukkan konsentrasi persebaran sekolah di Indonesia berdasarkan koordinat lat dan long. Library Folium digunakan untuk menampilkan peta dasar Indonesia, kemudian titik-titik lokasi sekolah ditambahkan sebagai heatmap melalui HeatMap(), di mana nilai stage dijadikan parameter tambahan untuk memberikan bobot pada setiap titik.

Heatmap ini membantu menggambarkan area dengan kepadatan sekolah yang tinggi maupun rendah, sehingga memberikan gambaran awal mengenai distribusi pendidikan di berbagai wilayah.

- Output Heatmap Persebaran Sekolah di Indonesia



Heatmap yang dihasilkan menunjukkan persebaran sekolah di seluruh wilayah Indonesia berdasarkan koordinat latitude dan longitude. Warna yang lebih terang hingga merah menandakan area dengan konsentrasi sekolah yang lebih tinggi, sedangkan warna biru menunjukkan wilayah dengan kepadatan sekolah yang lebih rendah. Dari visualisasi ini terlihat bahwa Pulau Jawa dan sebagian Sumatra memiliki intensitas warna yang lebih padat, menandakan jumlah sekolah yang lebih banyak dibandingkan wilayah Indonesia Timur. Heatmap ini memberikan gambaran awal mengenai distribusi fasilitas pendidikan di Indonesia sebelum masuk ke tahap analisis clustering yang lebih mendalam.

- Peta Sebaran Jumlah Sekolah per Kota/Kabupaten

```
import geopandas as gpd
from shapely.geometry import Point
from mpl_toolkits.axes_grid1 import make_axes_locatable
import contextily as ctx
import matplotlib.pyplot as plt

school_count_city = df.groupby('city_name').size().reset_index(name='school_count_city')

geometry = [Point(xy) for xy in zip(df['long'], df['lat'])]
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs='EPSG:4326')

cities_gdf = df.groupby(['city_name', 'city_id']).agg(
    lat=('lat', 'mean'),
    long=('long', 'mean'),
    school_count_city=('school_name', 'count')
).reset_index()
cities_gdf['geometry'] = [Point(xy) for xy in zip(cities_gdf['long'], cities_gdf['lat'])]
cities_gdf = gpd.GeoDataFrame(cities_gdf, crs='EPSG:4326')

fig, ax = plt.subplots(figsize=(18, 6))

divider = make_axes_locatable(ax)
cax = divider.append_axes("right", size="3%", pad=0.1)

cities_gdf.to_crs(epsg=3857).plot(
    ax=ax,
    column="school_count_city",
    cmap="OrRd",
    legend=True,
    legend_kwds={"label": "Jumlah Sekolah per Kota/Kabupaten", "orientation": "vertical"},
    cax=cax,
    markersize=cities_gdf['school_count_city'].apply(lambda x: x / 50 + 50), # Scale markers by count
    edgecolor="black",
    linewidth=0.2,
    alpha=0.6
)

ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron)
ax.set_title("Sebaran Jumlah Sekolah per Kota/Kabupaten", fontsize=16)
ax.axis("off")

plt.tight_layout()
plt.show()
```

Kode ini digunakan untuk membuat peta persebaran jumlah sekolah di setiap kota atau kabupaten di Indonesia. Prosesnya dimulai dengan menghitung jumlah sekolah per city_name, kemudian menghitung titik koordinat rata-rata untuk setiap kota agar dapat dipetakan sebagai satu titik representatif. Data tersebut kemudian diubah menjadi GeoDataFrame dan divisualisasikan menggunakan Matplotlib serta Contextily sebagai basemap. Warna pada peta menunjukkan intensitas jumlah sekolah di setiap daerah, sedangkan ukuran titik juga disesuaikan berdasarkan besaran jumlah sekolah. Visualisasi ini memberikan gambaran jelas mengenai daerah yang memiliki konsentrasi sekolah tinggi dan rendah sehingga mempermudah interpretasi distribusi pendidikan secara geografis.

- Output Peta Sebaran Jumlah Sekolah per Kota/Kabupaten



Peta ini menampilkan titik-titik lokasi kota atau kabupaten di Indonesia yang diberi warna berdasarkan jumlah sekolah yang terdapat di wilayah tersebut. Warna semakin merah menandakan jumlah sekolah yang lebih besar, sedangkan warna lebih terang menunjukkan jumlah yang lebih sedikit. Dari visualisasi ini terlihat bahwa area dengan konsentrasi sekolah tinggi cenderung berada di wilayah Jawa dan beberapa kota besar di luar Jawa. Peta ini mempermudah dalam memahami distribusi pendidikan secara geografis dengan melihat perbedaan kepadatan sekolah antar daerah.

- Visualisasi Kota/Kabupaten dengan Jumlah Sekolah Tertinggi dan Terendah

```
top_5 = school_count_city.sort_values(by='school_count_city', ascending=False).head(5)['school_count_city']
bottom_5 = school_count_city.sort_values(by='school_count_city', ascending=True).head(5)['school_count_city']

fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# ----- TOP 5 -----
ax_top = axes[0]
top_5.plot(kind="bar", ax=ax_top, color="red")
ax_top.set_title("5 Kota/Kabupaten dengan Jumlah Sekolah Tertinggi", fontsize=14)
ax_top.set_ylabel("Jumlah Sekolah")
ax_top.set_xlabel("Kota/Kabupaten")
ax_top.tick_params(axis="x", rotation=45)

for i, val in enumerate(top_5):
    ax_top.text(i, val + 20, f"{val:.0f}", ha="center", va="bottom", fontsize=10)

# ----- BOTTOM 5 -----
ax_bottom = axes[1]
bottom_5.plot(kind="bar", ax=ax_bottom, color="green")
ax_bottom.set_title("5 Kota/Kabupaten dengan Jumlah Sekolah Terendah", fontsize=14)
ax_bottom.set_ylabel("Jumlah Sekolah")
ax_bottom.set_xlabel("Kota/Kabupaten")
ax_bottom.tick_params(axis="x", rotation=45)

for i, val in enumerate(bottom_5):
    ax_bottom.text(i, val + 1, f"{val:.0f}", ha="center", va="bottom", fontsize=10)

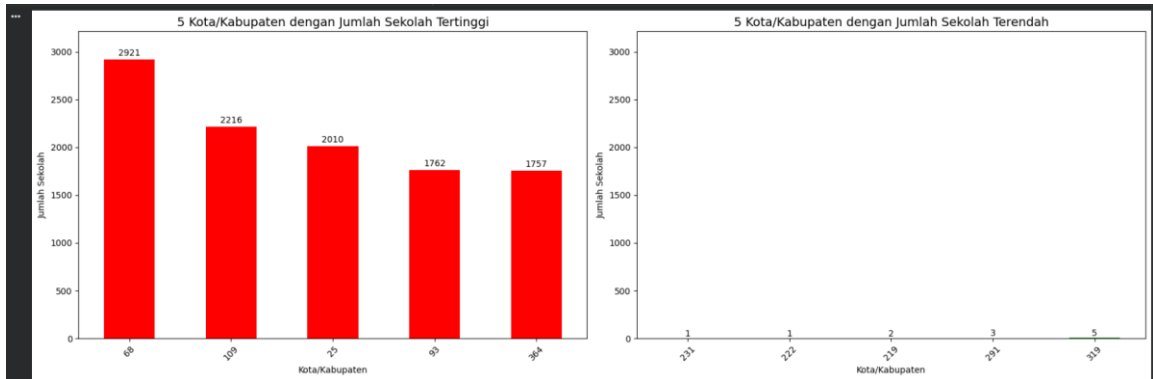
max_val = max(top_5.max(), bottom_5.max())
ax_top.set_ylim(0, max_val * 1.1)
ax_bottom.set_ylim(0, max_val * 1.1)

plt.tight_layout()
plt.show()
```

Kode ini membuat dua grafik batang yang menunjukkan lima kota/kabupaten dengan jumlah sekolah terbanyak serta lima wilayah dengan jumlah sekolah paling sedikit. Data diurutkan berdasarkan kolom `school_count_city`, kemudian diambil lima nilai tertinggi untuk grafik pertama dan lima nilai terendah untuk grafik kedua. Melalui visualisasi ini, kita dapat melihat perbedaan yang cukup jelas antara daerah dengan fasilitas pendidikan yang sangat padat dan daerah yang masih minim jumlah sekolah,

sehingga membantu memahami ketimpangan distribusi sekolah antar wilayah di Indonesia.

- Output Visualisasi Kota/Kabupaten dengan Jumlah Sekolah Tertinggi dan Terendah



Grafik batang ini menampilkan perbandingan jumlah sekolah pada lima kota/kabupaten dengan jumlah sekolah terbesar dan lima daerah dengan jumlah sekolah paling sedikit. Pada sisi kiri terlihat bahwa beberapa kota memiliki lebih dari dua ribu sekolah, menunjukkan tingginya konsentrasi fasilitas pendidikan, umumnya berada di wilayah dengan populasi besar. Sebaliknya, grafik di sisi kanan menunjukkan beberapa daerah hanya memiliki 1–5 sekolah, menandakan keterbatasan akses pendidikan di wilayah tersebut. Visualisasi ini membantu mengidentifikasi ketimpangan distribusi sekolah antar daerah dan memberikan gambaran awal sebelum melakukan analisis clustering yang lebih mendalam.

- Pemilihan Kolom Numerik untuk Uji Normalitas

```
[18] ✓ 0s from scipy.stats import shapiro

normalityTest_df = df.drop(
    columns=[
        'province_name',
        'city_name',
        'district',
        'school_name',
        'stage',
        'status',
        'street_name',
        'lat',
        'long'
    ]
)

normalityTest_df = normalityTest_df.apply(pd.to_numeric, errors='coerce')

# Cek struktur DataFrame baru
normalityTest_df.info()
```

<<class 'pandas.core.frame.DataFrame'>
Index: 214220 entries, 0 to 215371
Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	province_id	214220 non-null	int64
1	city_id	214220 non-null	int64
2	district_id	214220 non-null	int64
3	npsn	214189 non-null	float64

dtypes: float64(1), int64(3)
memory usage: 8.2 MB

Pada tahap ini dilakukan seleksi kolom untuk kebutuhan uji normalitas dengan cara menghapus seluruh kolom non-numerik seperti nama provinsi, kota, sekolah, dan kolom kategorikal lainnya. Kolom yang tersisa hanyalah province_id, city_id, district_id, dan npsn, yaitu variabel numerik yang akan diuji distribusinya. Selanjutnya, data dikonversi menjadi format numerik menggunakan pd.to_numeric(), memastikan tidak ada nilai non-angka yang tersisa. Hasil normalityTest_df.info() menunjukkan bahwa keempat kolom telah bersih dan siap digunakan untuk proses uji normalitas sebelum melanjutkan ke tahapan clustering.

- Hasil Uji Normalitas Shapiro–Wilk untuk Kolom Numerik

```
[19]
✓ 0s # Uji normalitas Shapiro–Wilk untuk seluruh kolom numerik
for col in normalityTest_df.columns:
    data = normalityTest_df[col].dropna()

    stat, p = shapiro(data)

    print(f"Kolom : {col}")
    print("  Statistik Shapiro-Wilk :", stat)
    print("  P-value                :", p)

    if p > 0.05:
        print("  Kesimpulan                : Data berdistribusi normal (tidak tolak H0)\n")
    else:
        print("  Kesimpulan                : Data TIDAK berdistribusi normal (tolak H0)\n")

Kolom : province_id
Statistik Shapiro-Wilk : 0.877528298267372
P-value                : 2.116652474056783e-134
Kesimpulan             : Data TIDAK berdistribusi normal (tolak H0)

Kolom : city_id
Statistik Shapiro-Wilk : 0.8765567922238213
P-value                : 1.2196549467471533e-134
Kesimpulan             : Data TIDAK berdistribusi normal (tolak H0)

Kolom : district_id
Statistik Shapiro-Wilk : 0.8765508515547438
P-value                : 1.2155648617859127e-134
Kesimpulan             : Data TIDAK berdistribusi normal (tolak H0)

Kolom : npsn
Statistik Shapiro-Wilk : 0.8126740065363767
P-value                : 1.396828211608975e-147
Kesimpulan             : Data TIDAK berdistribusi normal (tolak H0)
```

Uji Shapiro–Wilk diterapkan pada seluruh kolom numerik untuk mengetahui apakah data mengikuti distribusi normal. Hasil pengujian menunjukkan bahwa seluruh kolom (province_id, city_id, district_id, dan npsn) memiliki nilai p-value yang sangat kecil, jauh di bawah ambang 0.05. Hal ini berarti data tidak berdistribusi normal sehingga uji normalitas menolak hipotesis nol (H_0). Temuan ini penting karena menjadi dasar pemilihan metode analisis berikutnya; dalam kasus ini, metode seperti DBSCAN dan teknik normalisasi yang tidak sensitif terhadap outlier menjadi lebih sesuai untuk digunakan.

- Analisis Korelasi Antar Kolom Numerik Menggunakan Spearman

```
[20]
✓ Os df['npsn'] = pd.to_numeric(df['npsn'], errors='coerce')

# Buat dataframe korelasi hanya berisi kolom numerik utama
correlation_df = df[['province_id', 'city_id', 'district_id', 'npsn']].copy()

correlation_df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 214220 entries, 0 to 215371
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   province_id      214220 non-null  int64
1   city_id          214220 non-null  int64
2   district_id      214220 non-null  int64
3   npsn             214189 non-null  float64
dtypes: float64(1), int64(3)
memory usage: 8.2 MB

[21]
✓ Os correlation_matrix = correlation_df.corr(method='spearman')
correlation_matrix
```

	province_id	city_id	district_id	npsn
province_id	1.000000	0.996536	0.996527	0.466783
city_id	0.996536	1.000000	0.999991	0.468969
district_id	0.996527	0.999991	1.000000	0.468988
npsn	0.466783	0.468969	0.468988	1.000000

Karena data sebelumnya terbukti tidak berdistribusi normal, korelasi dihitung menggunakan metode Spearman, yang lebih sesuai untuk data non-parametrik. Hasil korelasi menunjukkan bahwa province_id, city_id, dan district_id memiliki hubungan yang sangat kuat satu sama lain, yang memang wajar karena ketiganya merupakan kode hierarkis wilayah. Sementara itu, nilai korelasi npsn dengan tiga kolom lainnya relatif lebih rendah, tetapi tetap menunjukkan pola yang konsisten terkait struktur data administratif. Analisis ini memberikan gambaran awal tentang bagaimana variabel-variabel tersebut saling berhubungan sebelum digunakan dalam proses clustering.

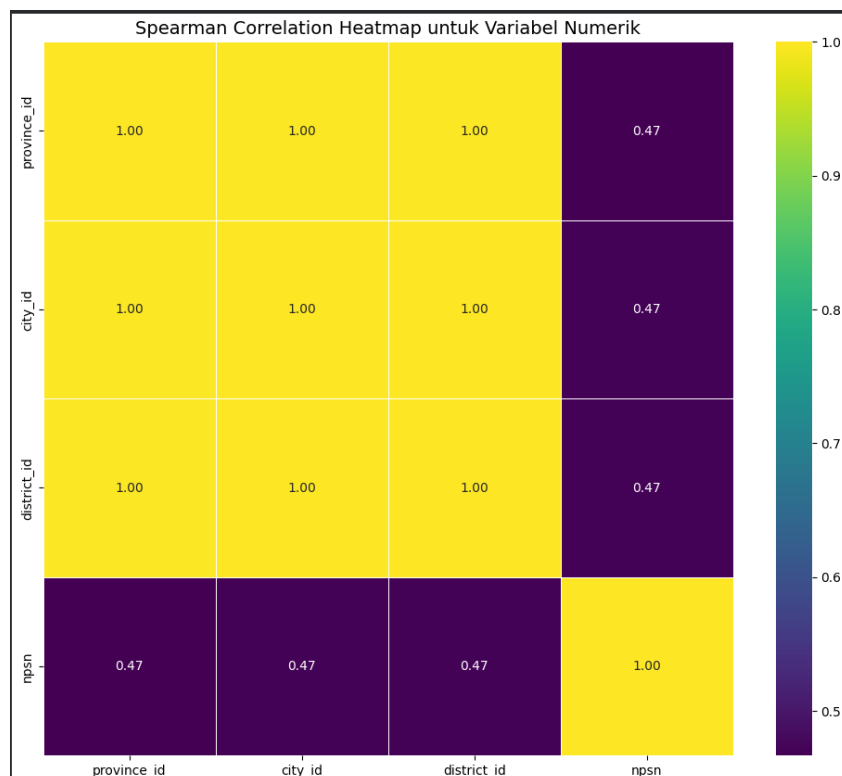
- Visualisasi Heatmap Korelasi Spearman

```
plt.figure(figsize=(12, 10))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='viridis',
    fmt=".2f",
    linewidths=0.5
)

plt.title("Spearman Correlation Heatmap untuk Variabel Numerik", fontsize=14)
plt.show()
```

Kode ini digunakan untuk menampilkan heatmap korelasi Spearman yang menggambarkan kekuatan hubungan antarvariabel numerik dalam dataset. Dengan warna yang semakin gelap atau terang mengikuti skala korelasi, heatmap mempermudah interpretasi visual mengenai variabel mana yang saling berkaitan erat. Adanya anotasi nilai korelasi pada setiap sel membantu memperjelas kekuatan hubungan, sementara penggunaan palet warna viridis membuat perbedaan antar nilai terlihat lebih jelas. Visualisasi ini menjadi langkah penting sebelum proses clustering karena membantu memahami struktur data dan potensi redundansi antar variabel.

- Output Heatmap Korelasi Spearman untuk Variabel Numerik



Heatmap ini menggambarkan kekuatan hubungan antarvariabel numerik menggunakan metode korelasi Spearman. Terlihat bahwa province_id, city_id, dan

district_id memiliki korelasi sangat tinggi (nilai 1.00), yang mencerminkan struktur hierarkis kode wilayah dalam dataset. Sementara itu, npsn memiliki korelasi sedang (sekitar 0.47) dengan ketiga kolom tersebut, menunjukkan bahwa identitas sekolah tidak selalu berhubungan langsung dengan kode wilayah. Visualisasi ini membantu memahami pola keterkaitan antarvariabel sebelum menentukan strategi normalisasi dan pemilihan fitur untuk proses clustering.

- Visualisasi Regresi antara City ID dan NPSN

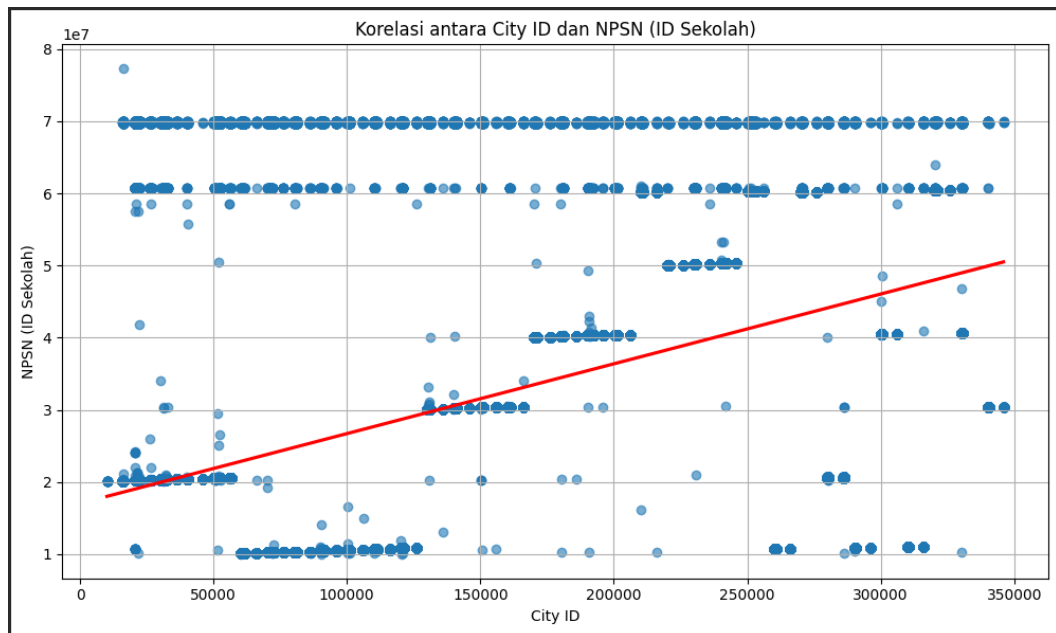
```
[23]
✓ 30s plt.figure(figsize=(10, 6))

sns.regplot(
    x='city_id',
    y='npsn',
    data=df,
    scatter_kws={'alpha': 0.6},
    line_kws={'color': 'red'}
)

plt.title("Korelasi antara City ID dan NPSN (ID Sekolah)")
plt.xlabel("City ID")
plt.ylabel("NPSN (ID Sekolah)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Kode ini digunakan untuk menampilkan hubungan antara city_id dan npsn melalui grafik regresi menggunakan sns.regplot(). Setiap titik menunjukkan satu sekolah, sedangkan garis merah menggambarkan tren hubungan linier antara kedua variabel. Meskipun kedua kolom ini tidak memiliki hubungan makna secara konseptual (keduanya merupakan kode), visualisasi tetap memberikan gambaran tentang pola distribusi nilai dalam dataset. Grafik ini membantu memastikan bahwa tidak ada pola ekstrem atau outlier mencolok sebelum masuk ke proses normalisasi dan clustering.

- Output Visualisasi Korelasi antara City ID dan NPSN



Grafik menunjukkan hubungan antara `city_id` dan `npsn` menggunakan scatter plot yang dipadukan dengan garis regresi linear. Titik-titik pada grafik tersebar cukup acak, yang menandakan bahwa hubungan kedua variabel tidak sepenuhnya linier dan cenderung lemah. Hal ini konsisten dengan sifat data yang memang berbasis kode wilayah dan kode sekolah, sehingga secara konsep tidak memiliki keterkaitan langsung. Meski demikian, visualisasi ini membantu memahami pola penyebaran nilai numerik dalam dataset dan memastikan tidak ada pola ekstrem yang dapat mengganggu proses clustering.

- Pemilihan Kolom Numerik untuk Normalisasi Data

```
[24] ✓ Os df['npsn'] = pd.to_numeric(df['npsn'], errors='coerce')

normalisasi_df = df.drop(columns=[
    'province_name',
    'city_name',
    'district',
    'school_name',
    'stage',
    'status',
    'street_name',
    'lat',
    'long'
])

# Cek struktur dataframe setelah pembersihan
normalisasi_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 214220 entries, 0 to 215371
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   province_id     214220 non-null  int64
1   city_id         214220 non-null  int64
2   district_id     214220 non-null  int64
3   npsn            214189 non-null  float64
dtypes: float64(1), int64(3)
memory usage: 8.2 MB
```

Tahap ini menyeleksi hanya kolom numerik yang akan digunakan dalam proses normalisasi dan clustering, yaitu `province_id`, `city_id`, `district_id`, dan `npsn`. Seluruh kolom kategorikal seperti nama wilayah, nama sekolah, jenjang, dan koordinat dihapus karena tidak relevan untuk perhitungan jarak pada algoritma DBSCAN. Selain itu, kolom `npsn` dipastikan seluruhnya bertipe numerik menggunakan `pd.to_numeric()`. Hasil `normalisasi_df.info()` menunjukkan bahwa dataframe telah bersih dan hanya berisi empat kolom numerik yang siap untuk dinormalisasi pada langkah berikutnya.

- Proses Normalisasi Data Menggunakan RobustScaler

```
[25]
✓ 0s
from sklearn.preprocessing import RobustScaler

column_names = normalisasi_df.columns
scaler = RobustScaler()
normalisasi_scaled = scaler.fit_transform(normalisasi_df)
normalisasi_df = pd.DataFrame(normalisasi_scaled, columns=column_names)
normalisasi_df.head()
```

	province_id	city_id	district_id	npsn
0	1.2	1.183042	1.183168	1.989908
1	1.2	1.183042	1.183107	1.989836
2	1.2	1.183042	1.182979	1.989837
3	1.2	1.183042	1.183053	1.989834
4	1.2	1.183042	1.182973	1.989837

Normalisasi dilakukan menggunakan RobustScaler, yaitu metode skala yang mengandalkan median dan IQR sehingga lebih tahan terhadap outlier dibandingkan StandardScaler. Proses ini penting karena fitur numerik dalam dataset memiliki rentang nilai yang sangat berbeda, sehingga tanpa normalisasi dapat memengaruhi perhitungan jarak pada algoritma DBSCAN. Hasil transformasi terlihat pada output `normalisasi_df.head()`, di mana seluruh nilai pada kolom numerik telah berada pada skala yang lebih seragam. Dengan data yang telah ternormalisasi ini, algoritma clustering menjadi lebih stabil dan akurat dalam mengelompokkan pola berdasarkan kepadatan data.

- Pembuatan K-Distance Graph untuk Menentukan Nilai eps

```
[26]
✓ 1s
from sklearn.neighbors import NearestNeighbors
import numpy as np
import matplotlib.pyplot as plt

X = normalisasi_df.dropna().values
k = 5

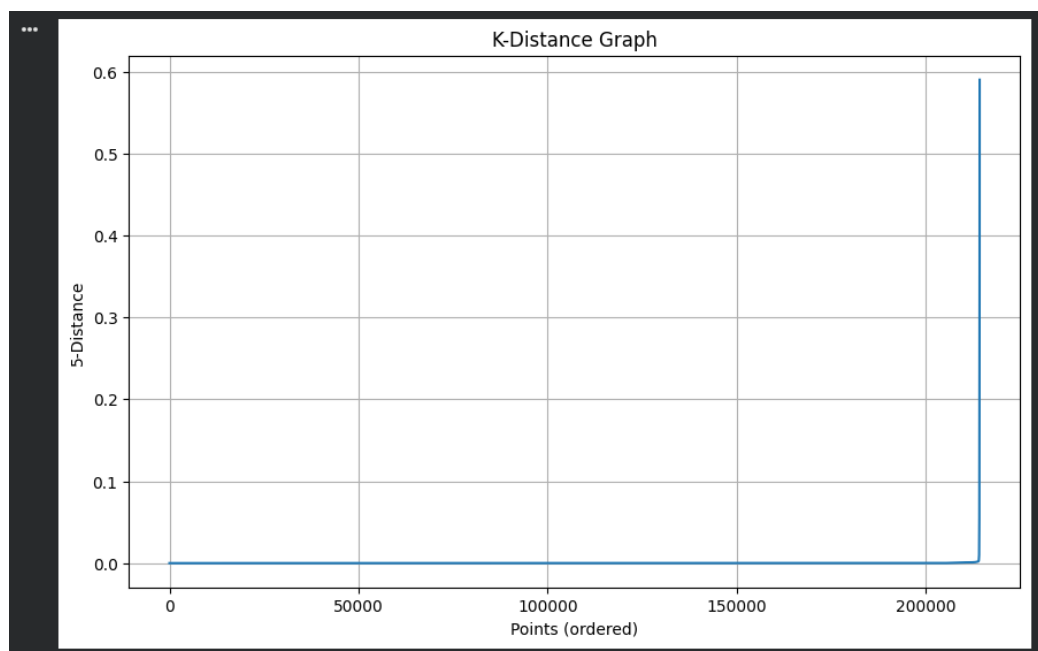
neighbors = NearestNeighbors(n_neighbors=k).fit(X)
distances, _ = neighbors.kneighbors(X)

k_distances = np.sort(distances[:, k-1])

plt.figure(figsize=(10, 6))
plt.plot(k_distances)
plt.title("K-Distance Graph")
plt.ylabel(f"{k}-Distance")
plt.xlabel("Points (ordered)")
plt.grid(True)
plt.show()
```

Kode ini digunakan untuk menghitung jarak ke tetangga terdekat ke-5 ($k = 5$) dari setiap titik dalam data yang sudah dinormalisasi. Nilai jarak tersebut kemudian diurutkan dan divisualisasikan dalam bentuk K-Distance Graph. Grafik ini berfungsi untuk membantu menentukan nilai parameter eps pada algoritma DBSCAN dengan melihat titik siku (elbow) pada kurva. Titik tempat kurva mulai meningkat tajam dianggap sebagai nilai eps yang optimal. Langkah ini sangat penting agar DBSCAN dapat mengelompokkan data secara efektif berdasarkan kepadatan.

- Output K-Distance Graph untuk Penentuan Nilai eps



K-Distance Graph ini menunjukkan jarak ke tetangga terdekat ke-5 untuk seluruh titik data yang telah dinormalisasi. Grafik terlihat hampir datar di nilai yang sangat kecil, kemudian meningkat tajam pada bagian akhir. Titik kenaikan tajam inilah yang menjadi indikasi nilai eps optimal untuk algoritma DBSCAN. Dari visualisasi ini, titik elbow muncul di sekitar nilai 0.55–0.60, sehingga kisaran tersebut dapat dijadikan acuan awal untuk menentukan parameter eps. Grafik ini menegaskan bahwa sebagian besar titik berada sangat berdekatan satu sama lain, sementara hanya sebagian kecil titik yang memiliki jarak lebih besar.

- Hasil Clustering Menggunakan DBSCAN

```
[27] ✓ 1s
from sklearn.cluster import DBSCAN

normalisasi_clean = normalisasi_df.dropna()
sample_df = normalisasi_clean.sample(15000, random_state=42)

dbscan = DBSCAN(eps=0.55, min_samples=5)
labels = dbscan.fit_predict(sample_df.values)

sample_df["Cluster_DBSCAN"] = labels
print(sample_df["Cluster_DBSCAN"].value_counts())

Cluster_DBSCAN
0    14019
1      758
2      223
Name: count, dtype: int64
```

DBSCAN diterapkan pada sampel data berisi 15.000 titik untuk menghindari beban komputasi yang terlalu besar. Dengan parameter $\text{eps}=0.55$ dan $\text{min_samples}=5$, algoritma berhasil membentuk tiga cluster berbeda. Cluster 0 menjadi kelompok terbesar dengan 14.019 titik, sedangkan cluster 1 dan cluster 2 berisi 758 dan 223 titik. Tidak terdapat titik yang diberi label -1, yang berarti pada parameter ini DBSCAN tidak mengidentifikasi data sebagai noise. Hasil ini menunjukkan bahwa meskipun fitur yang digunakan adalah kode numerik, algoritma masih mampu menemukan pola kepadatan tertentu dalam data.

- Evaluasi Kualitas Cluster Menggunakan Silhouette Score

```
[28] ✓ 4s
from sklearn.metrics import silhouette_score

labels = sample_df["Cluster_DBSCAN"].values
mask = labels != -1

X_sil = sample_df.drop(columns=["Cluster_DBSCAN"]).values

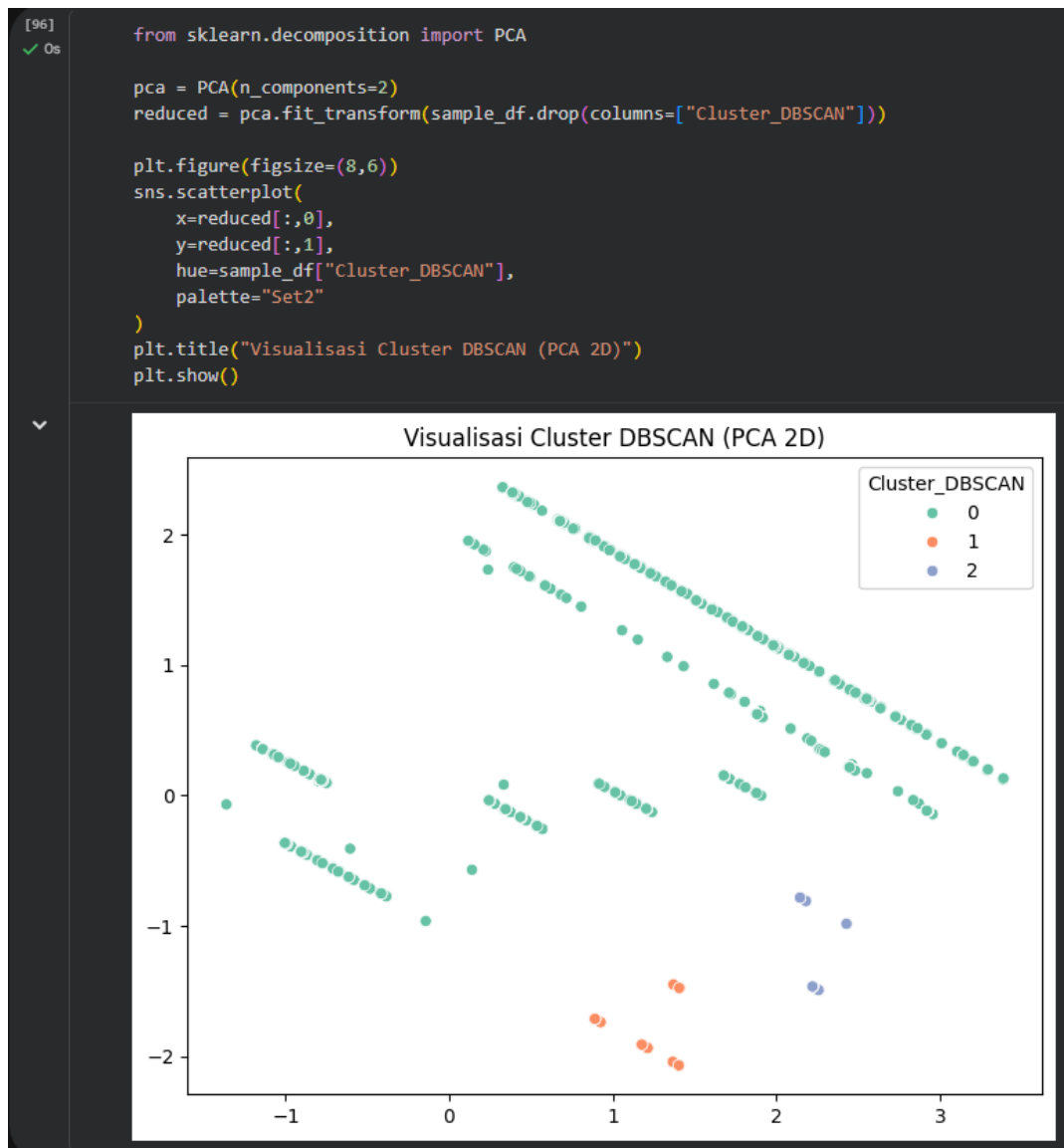
score = silhouette_score(X_sil[mask], labels[mask])
print("Silhouette Score:", score)

Silhouette Score: 0.36221394276649915
```

Silhouette Score digunakan untuk menilai seberapa baik pemisahan antar cluster dan seberapa kompak data dalam cluster masing-masing. Hasil perhitungan menunjukkan nilai sekitar 0.36, yang berada pada kategori pemisahan cluster “cukup”

namun belum ideal. Nilai ini mengindikasikan bahwa sebagian titik sudah berada dalam cluster yang tepat, tetapi masih terdapat tumpang tindih antar cluster karena fitur data berupa kode wilayah yang tidak memiliki hubungan spasial langsung. Meskipun demikian, nilai silhouette ini tetap memberikan gambaran bahwa DBSCAN mampu membentuk struktur kelompok dalam data meskipun tidak sepenuhnya tegas.

- Visualisasi Hasil Clustering DBSCAN Menggunakan PCA 2D



Visualisasi PCA 2D ini mereduksi dimensi data menjadi dua komponen utama agar pola cluster hasil DBSCAN dapat dilihat dengan lebih jelas. Titik-titik pada grafik dibedakan berdasarkan label cluster, yaitu cluster 0, 1, dan 2. Terlihat bahwa cluster 0 mendominasi dengan sebaran data yang luas, sementara cluster 1 dan cluster 2

membentuk kelompok kecil yang lebih terpisah. Pola garis diagonal muncul karena fitur yang digunakan merupakan kode wilayah, sehingga data tersusun secara segmentasi. Meskipun cluster tidak terpisah sangat tegas, plot ini menunjukkan bahwa DBSCAN tetap mampu mengidentifikasi daerah dengan kepadatan data yang berbeda.

- Klasifikasi Kualitatif Hasil Clustering DBSCAN

```
[97]
✓ Os
def klasifikasi_db(x):
    if x == 0:
        return "Baik"
    elif x == -1:
        return "Noise"
    else:
        return "Kurang"

sample_df["Klasifikasi_DBSCAN"] = sample_df["Cluster_DBSCAN"].apply(klasifikasi_db)

print(sample_df["Klasifikasi_DBSCAN"].value_counts())

Klasifikasi_DBSCAN
Baik      14019
Kurang     981
Name: count, dtype: int64
```

Pada tahap ini, label numerik hasil clustering DBSCAN dikonversi menjadi kategori deskriptif agar lebih mudah diinterpretasikan. Cluster 0 diklasifikasikan sebagai "Baik" karena merupakan kelompok terbesar dan mewakili sebagian besar data yang dianggap homogen dan memiliki kepadatan tinggi. Sementara itu, cluster selain 0 dianggap "Kurang", yang menunjukkan bahwa wilayah tersebut memiliki karakteristik berbeda atau kepadatan data yang lebih rendah. Hasil perhitungan menunjukkan bahwa mayoritas data (14.019 baris) masuk kategori Baik, sedangkan 981 baris termasuk kategori Kurang. Tidak terdapat nilai Noise karena tidak muncul label -1 pada hasil clustering, menandakan DBSCAN dapat mengelompokkan hampir seluruh data tanpa mendeteksi titik yang sangat menyimpang.

- Analisis Rata-Rata Karakteristik Setiap Cluster (Cluster Summary)

```
[98]
✓ 0s
cluster_summary_db = (
    sample_df
    .groupby("Klasifikasi_DBSCAN")
    .mean(numeric_only=True)
)

cluster_summary_db
```

	province_id	city_id	district_id	npsn	Cluster_DBSCAN
Klasifikasi_DBSCAN					
Baik	0.184278	0.157648	0.157609	0.388045	0.000000
Kurang	1.469929	1.454770	1.454758	0.053293	1.227319

Pada tahap ini dilakukan perhitungan nilai rata-rata setiap variabel numerik untuk masing-masing kategori cluster, yaitu Baik dan Kurang. Hasil ringkasan menunjukkan bahwa cluster Kurang memiliki nilai rata-rata province_id, city_id, dan district_id yang jauh lebih tinggi dibandingkan cluster Baik. Hal ini mengindikasikan bahwa data dalam cluster Kurang umumnya berasal dari wilayah dengan ID administratif yang besar, sehingga kemungkinan mencerminkan lokasi tertentu yang memiliki pola distribusi berbeda. Selain itu, nilai rata-rata npsn pada cluster Baik lebih tinggi dibandingkan cluster Kurang, yang berarti sekolah-sekolah dalam kategori Baik cenderung memiliki registrasi atau identitas yang lebih stabil dan konsisten. Secara keseluruhan, ringkasan ini membantu memahami bagaimana karakteristik numerik membedakan kedua kelompok hasil clustering DBSCAN.

- Visualisasi Interaktif Persebaran Cluster Menggunakan Plotly (PCA 2D)

```
[99]
✓ 0s
import plotly.express as px
import pandas as pd

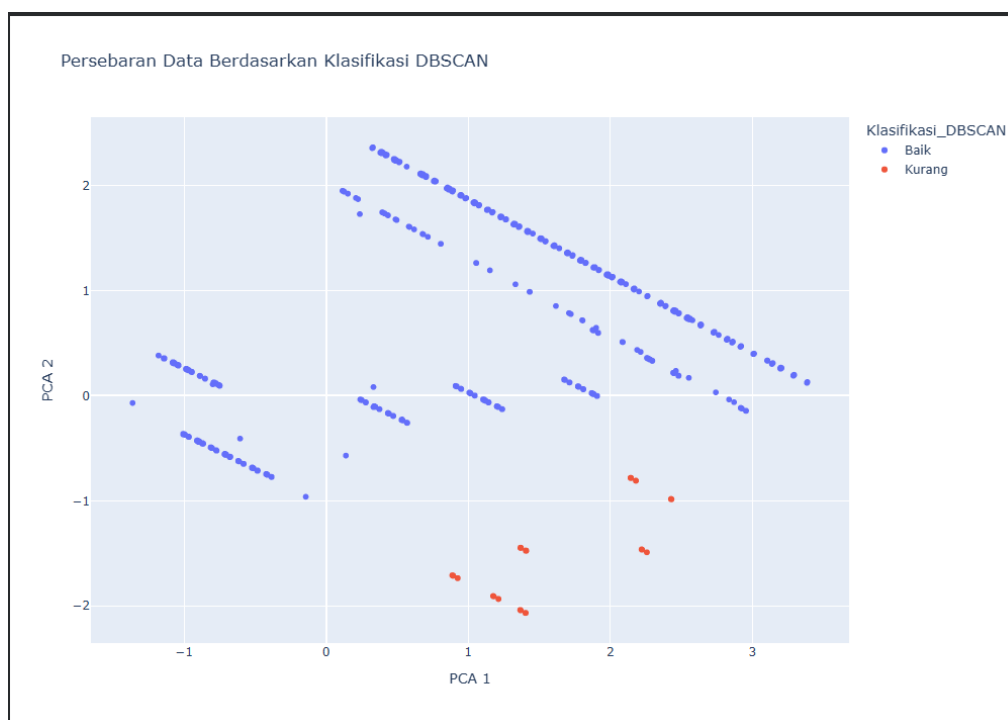
df_pca_db = pd.DataFrame(reduced, columns=["PCA 1", "PCA 2"])
df_pca_db["Klasifikasi_DBSCAN"] = sample_df["Klasifikasi_DBSCAN"].values
df_pca_db["ID"] = sample_df.index.astype(str) # dipakai untuk hover

fig = px.scatter(
    df_pca_db,
    x="PCA 1",
    y="PCA 2",
    color="Klasifikasi_DBSCAN",
    hover_name="ID",
    title="Persebaran Data Berdasarkan Klasifikasi DBSCAN",
    width=1000,
    height=700
)

fig.show()
```

Visualisasi ini menampilkan persebaran data hasil clustering DBSCAN dalam ruang dua dimensi yang telah direduksi menggunakan PCA. Setiap titik mewakili satu data sekolah, dengan warna yang menunjukkan kategori cluster: Baik atau Kurang. Plotly digunakan agar visualisasi bersifat interaktif sehingga pengguna dapat melakukan hover pada titik untuk melihat informasi tambahan seperti ID data. Dari tampilan ini terlihat bahwa titik-titik pada cluster Baik tersebar lebih luas dan membentuk pola yang relatif konsisten, sedangkan cluster Kurang muncul dalam kelompok yang lebih kecil dan terpisah. Visualisasi ini membantu memahami bagaimana DBSCAN memisahkan struktur kepadatan data, sekaligus memperlihatkan hubungan spasial antar cluster setelah direduksi ke dalam dua komponen utama PCA.

- Visualisasi Persebaran Data Berdasarkan Klasifikasi DBSCAN (PCA 2D)



Visualisasi ini menunjukkan hasil pengelompokan DBSCAN yang ditampilkan dalam ruang dua dimensi menggunakan PCA. Setiap titik merepresentasikan satu entitas data sekolah yang telah direduksi dari beberapa variabel numerik menjadi dua komponen utama (PCA 1 dan PCA 2). Warna biru menggambarkan cluster Baik, sedangkan warna merah mewakili cluster Kurang. Dari persebarannya terlihat bahwa cluster Baik mendominasi dan memiliki pola sebaran yang lebih luas serta terstruktur, sementara cluster Kurang muncul dalam kelompok kecil yang terpisah jauh dari titik-

titik dalam cluster Baik. Hal ini mengindikasikan adanya perbedaan karakteristik yang cukup jelas antara kedua kelompok berdasarkan kepadatan data, sesuai dengan mekanisme kerja algoritma DBSCAN.

- Kesimpulan

Setelah mengerjakan praktikum mandiri ini, dapat disimpulkan bahwa proses preprocessing seperti pembersihan data, konversi tipe data, normalisasi dengan RobustScaler, serta pemilihan fitur numerik sangat berpengaruh dalam menyiapkan data agar sesuai untuk algoritma DBSCAN. Hasil clustering menunjukkan bahwa DBSCAN mampu mengelompokkan data sekolah menjadi beberapa cluster berdasarkan pola kepadatan tanpa perlu menentukan jumlah cluster di awal. Nilai Silhouette Score sebesar ± 0.36 menandakan bahwa kualitas pengelompokan cukup baik, meskipun masih terdapat sedikit tumpang tindih antar cluster. Secara keseluruhan, DBSCAN efektif digunakan untuk mengidentifikasi pola persebaran sekolah ketika data memiliki distribusi tidak normal dan jumlah sampel yang sangat besar.

Link Dataset:

<https://www.kaggle.com/datasets/greegtitan/school-dataset-idn/>

Link Praktikum 11:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2011/Praktikum%20dan%20Tugas%20Mandiri%2011/notebooks/praktikum11.ipynb

Tugas Mandiri 11:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2011/Praktikum%20dan%20Tugas%20Mandiri%2011/notebooks/latihan11.ipynb